Modèles algorithmiques fondamentaux sur structures à accès direct

Thème principal : Initiation à l'algorithmique et patrons de conception (Design Patterns) sur structures à accès direct

Synthèse exécutive : Ce cours magistral pose les fondements méthodologiques indispensables de la conception algorithmique en BUT 1 Informatique (R1.01). Avant d'écrire la moindre ligne de code ou de pseudo-code, l'informaticien doit impérativement analyser le contrat d'interface et identifier le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours formalise trois grands modèles algorithmiques canoniques et réutilisables applicables sur des structures à accès direct (tableaux, chaînes de caractères) : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence (avec ses deux variantes stratégiques). Une rigueur absolue est exigée sur le respect des bornes d'indices (de 0 à N-1 pour éviter les erreurs 'out of bounds'), sur l'adoption du principe du pessimisme algorithmique, ainsi que sur l'obligation d'intégrer une double condition d'arrêt pour tout algorithme de recherche.

1. Typologie et Analyse Préalable des Mécanismes d'Accès aux Données

L'étape zéro de toute conception algorithmique consiste à analyser le contrat d'interface offert par la structure de données sous-jacente. Si la structure propose un accès direct (comme un tableau statique ou une chaîne de caractères), chaque case mémoire est immédiatement adressable par son indice i en temps constant O(1), sans parcourir les éléments précédents. En revanche, un flux séquentiel (tel qu'un fichier texte brut, un flux CSV lu ligne par ligne, ou une liste chaînée simple) n'expose que trois opérations primitives : (1) se positionner en début de flux, (2) lire l'élément suivant, et (3) tester si la fin du flux est atteinte. Il est matériellement et logiquement impossible de sauter directement à la 50ème ligne sans avoir lu les 49 premières. Appliquer une notation par indice de type T[i] sur un fichier séquentiel constitue une erreur conceptuelle critique qui invalide intégralement la solution proposée.

- Contrat d'interface et analyse de structure
- Accès direct (Tableaux, Chaînes de caractères)
- Accès séquentiel (Fichiers texte, Flux CSV)
- Accès dynamique (Structures hybrides)

Cas concret enseignant : *Le professeur insiste lourdement sur un piège récurrent en évaluation : rédiger un algorithme qui utilise un accès par indice 'T[i]' pour traiter un fichier texte ou un fichier CSV brut. Comme le système d'exploitation ne fournit pour ces fichiers qu'un curseur séquentiel avance-ligne, l'algorithme est sémantiquement faux et conduit directement à l'attribution de la note zéro sur la question.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique dès lors qu'aucune exclusion ni filtrage n'est permis au sein du domaine d'exploration. Il répond à des problématiques telles que : afficher toutes les valeurs d'un tableau, initialiser chaque case à zéro, ou convertir chaque caractère d'une chaîne en majuscule. Comme la taille du domaine à parcourir est fixée dès le départ, la structure itérative appropriée est la boucle bornée 'Pour'. En pseudo-code normalisé, pour un tableau de taille N, l'indice i démarre à 0 et termine impérativement à N - 1. Le traitement est exécuté de manière indifférenciée et systématique à chaque tour de boucle sans test préalable. Le piège classique des étudiants est de faire varier l'indice jusqu'à N, déclenchant une exception critique de dépassement d'indice (out of bounds).

- Exhaustivité du parcours (100% des éléments)
- Utilisation obligatoire de la boucle bornée (Pour / For)
- Gestion stricte des bornes d'indices (de 0 à N - 1)
- Prévention du dépassement de capacité (Out of Bounds)

Cas concret enseignant : *Exemple d'illustration vu en amphithéâtre : 'Afficher l'intégralité d'un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente chez les étudiants débutants consiste à écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat ('out of bounds') lors de la tentative de lecture de la case mémoire T[N] qui n'appartient pas au tableau.*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionnel

Le parcours complet avec traitement conditionné conserve l'exploration exhaustive du modèle précédent, mais encadre l'action de traitement par une condition de filtrage. Il est utilisé pour compter des occurrences particulières (ex: compter le nombre de voyelles dans un texte), afficher uniquement un sous-ensemble de données (ex: imprimer les nombres pairs d'un tableau), ou effectuer des calculs cumulatifs ciblés. La condition peut être simple (ex: T[i] > 0), complexe (ex: T[i] > 0 ET T[i] mod 2 = 0), ou porter sur une comparaison de valeurs contiguës (ex: T[i] > T[i-1]). Dans ce dernier cas, la boucle doit démarrer à i = 1 (au lieu de 0) pour empêcher l'évaluation de T[-1] qui provoquerait une erreur mémoire.

- Inspection globale et traitement sélectif
- Imbrication d'une structure alternative (Si ... Alors) dans une boucle bornée
- Prédicats logiques simples et complexes
- Ajustement des bornes en cas de comparaison avec les voisins (i-1 ou i+1)

Cas concret enseignant : *L'analogie de la chaîne de montage en usine donnée par le professeur : Sur une chaîne de fabrication, chaque pièce défile obligatoirement devant l'inspecteur qualité (parcours complet d'inspection de toutes les pièces), mais l'inspecteur n'appose le coup de tampon de rebut (action de traitement) que si et seulement si la pièce présente un défaut spécifique (traitement conditionné).*

4. Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

À l'inverse des parcours complets, la recherche de première occurrence s'interrompt dès que le premier élément satisfaisant le critère est localisé. L'esprit de conception repose sur le principe du pessimisme : on initialise un booléen 'trouve' à 'Faux' avant d'engager l'exploration. Tout algorithme de recherche valide DOIT obligatoirement intégrer deux conditions d'arrêt : (1) le succès de la recherche, (2) l'épuisement de la collection. Deux variantes structurelles existent : la variante 1 (rupture immédiate, idéale pour récupérer la position exacte) et la variante 2 (Tant que global, plus simple à structurer syntaxiquement avec un test en tête de boucle mais incrémentant l'indice une dernière fois).

- Pessimisme algorithmique (initialisation de trouve à Faux)
- Itération non bornée (Boucle Tant que / While)
- Double condition d'arrêt obligatoire (Succès OU Fin de structure)
- Variante 1 : Rupture immédiate (Indice exact conservé)
- Variante 2 : Tant que global (Test unique en tête)

Cas concret enseignant : *La règle d'or du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans la structure, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle qu'un algorithme de recherche présenté en examen avec une seule condition d'arrêt recevra systématiquement la note zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, tout à fait. Une structure à accès dynamique propose à la fois des mécanismes d'accès séquentiel et d'accès direct. Par conséquent, il est parfaitement possible d'appliquer des modèles d'accès direct en exploitant ces primitives.

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : La variante 1 (rupture immédiate) est indispensable si l'on a besoin de conserver la position indicée exacte de l'élément trouvé, car l'indice i s'arrête pile sur la cible. La variante 2 (tant que global) offre une syntaxe plus propre et épurée en tête de boucle, idéale lorsque seule l'existence booléenne de l'élément nous intéresse.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionnel dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Il faut impérativement réajuster les bornes de la boucle. Si l'on compare l'élément courant à son prédécesseur (i-1), la boucle ne peut pas démarrer à l'indice 0 (sous peine d'évaluer T[-1] et de provoquer un crash mémoire), mais doit obligatoirement démarrer à l'indice 1.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser la recherche de première occurrence (qui s'arrête au premier trouvé), mais le 'Parcours complet avec traitement conditionné' qui inspectera toutes les cases du tableau et déclenchera l'enregistrement de l'indice chaque fois que la condition T[i] = 5 sera vérifiée.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en explicitant le modèle retenu et la gestion des bornes.

Démarche de résolution :
Étape 1 : Analyse du problème et sélection du modèle. On doit inspecter l'intégralité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle algorithmique approprié est donc le 'Parcours complet avec traitement conditionné' (Modèle 2). Étape 2 : Détermination des bornes d'indices. En convention informatique standard (indexation à zéro), pour un tableau de N éléments, la première case est à l'indice 0 et la dernière à l'indice N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Choix de la structure de contrôle. Le nombre d'itérations est connu avant d'entrer dans la boucle -> Utilisation d'une boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Écriture du filtre conditionnel et du traitement. À chaque itération i, évaluer 'Si T[i] > 0 Alors' puis déclencher l'action 'Afficher(T[i])'.

```
Algorithme AffichagePositifs
// CONSTANTES
// (Aucune constante nécessaire)

// VARIABLES
// T : Tableau d'entiers
// N : Entier (taille du tableau)
// i : Entier (variable de boucle)

// TRAITEMENTS
// (clavier) >> saisir >> N, T
Début
    Pour i allant de 0 à N - 1 Faire
        Si T[i] > 0 Alors
            Afficher(T[i])
        Fin Si
    Fin Pour
Fin
```

Résultat : L'algorithme garantit un parcours exhaustif sans risque de dépassement de capacité (l'itération s'arrête exactement à N-1) et applique le filtre sélectif demandé de manière optimale.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique devez-vous utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La recherche de première occurrence (Modèle 3).

Q : Quel est le risque critique si l'on écrit une boucle 'Pour i allant de 0 à N' sur un tableau de N éléments indicé à partir de 0 ?

R : Une erreur de dépassement de capacité mémoire (exception 'out of bounds') lors de la tentative d'accès à la case inexistante T[N].

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'un algorithme de recherche de première occurrence et quelles sont-elles ?

R : Deux conditions d'arrêt minimales : (1) Le succès de la recherche (l'élément est trouvé) et (2) l'échec par épuisement complet de la collection (la fin de structure est atteinte).